

DOCUMENTAZIONE DI PROGETTO

Team D13
TASK R1

1. ORGANIZZAZIONE

1.1 STRUTTURA DEL TEAM

Sabino Gabriele Breglia	s.breglia@studenti.unina.it	DE9000097
Marcello Esposito	marcello.esposito6@studenti.unina.it	M63001768

URL della repository di partenza:

https://github.com/Testing-Game-SAD-2023/A13/tree/D13_TaskR1_UploadRobot

URL della repository del progetto svolto:

https://github.com/sabinogb/R1-task/tree/D13_TaskR1R2_UploadRobot_CaricamentoSuggerimenti_MigrazioneMongoDB

1.2 DEFINIZIONE DEGLI OBIETTIVI DI PROGETTO

Task assegnato

Il task da svolgere consiste nel **migliorare la robustezza della funzionalità di upload** delle classi e dei relativi unit test. In particolare, i problemi principali che sono stati evidenziati all'assegnazione del task sono due:

1. in caso di upload fallito, sul volume condiviso possono persistere dei file da scartare,
2. in caso di upload fallito, all'amministratore non viene comunicato alcun messaggio di errore che indica in maniera dettagliata la causa dell'errore.

Approccio di sviluppo seguito

Sono stati seguiti i principi **Agile**, con particolare riferimento all'**XP** che si adatta molto bene alla natura di questo task, essendo composto da numerosi piccoli subtask.

Più nel dettaglio, tra i principi cardine seguiti, si riportano:

- **Pair Programming**: è vantaggioso, dal momento che il team composto da due studenti, e permette inoltre di ridurre il tempo di validazione delle nuove modifiche, dato che queste vengono validate da entrambi in tempo reale,
- **Small Increments**: data la natura del task, il modo più semplice per effettuare modifiche e poter tornare ad uno stato consistente del sistema è stato quello di effettuare tanti piccoli incrementi (commit),
- **Testing**: per quanto non si possa parlare di TDD dato che non sono stati formalizzati i

test né tantomeno automatizzati, le nuove modifiche nel sistema sono state continuamente testate per garantire il corretto funzionamento ad ogni nuovo incremento, o comunque un insieme piccolo di incrementi,

- **Simple Design**: si è cercato di mantenere le componenti quanto più semplici possibili,
- **Continuous Refactoring**: di volta in volta è stato continuamente valutato il caso di rifattorizzare un metodo o un servizio, anche grazie all'utilizzo di strumenti come **SonarQube**,
- **Timeboxing**: non sempre la valutazione dei tempi è stata precisa, dovendo rimandare alcune modifiche alla iterazione successiva.

Gestione del progetto

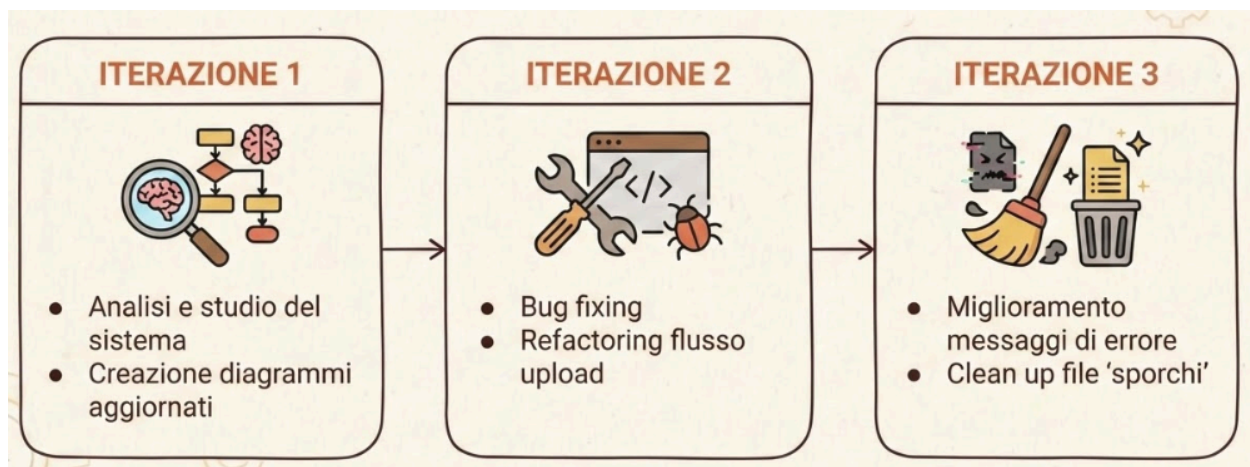
Seguito il modello di sviluppo del **framework SCRUM** data la natura di questo esame.

Nella prima iterazione è stato principalmente analizzato il sistema per definire il product backlog complessivo.

Successivamente, per ciascuna iterazione è stato deciso il **tema** per lo Sprint backlog, così da lavorare su requisiti della stessa tipologia.

I vari subtask, infatti, sono stati raggruppati per tipologia (refactoring, fix, documentazione, nuove funzionalità).

Il processo di sviluppo ha poi avuto anche diverse analogie con l'**UP**: è stata effettuata una prima fase di analisi e stima dei costi, per poi passare allo sviluppo dei requisiti Core (**bug fixing** e **refactoring**), seguiti dall'implementazione delle funzionalità rimanenti (es. error response più parlante) ed il rilascio del sistema tramite pull request.



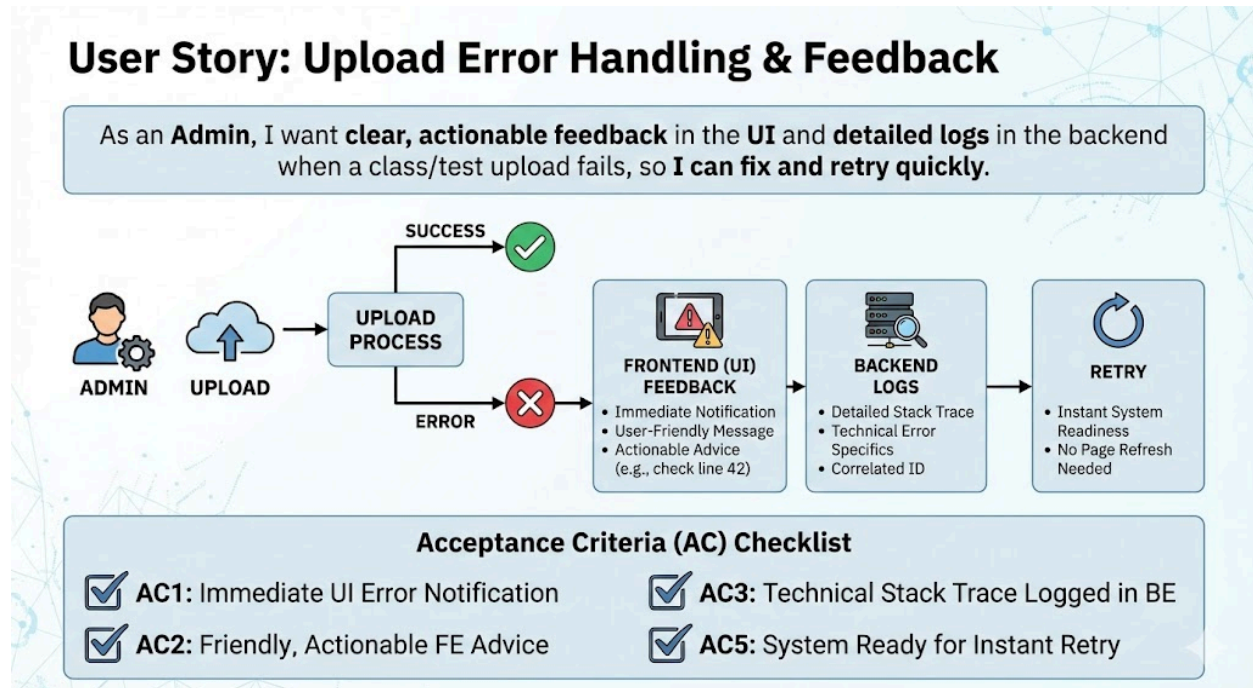
2. ANALISI DEI REQUISITI PER IL TASK ASSEGNATO

2.1 Analisi dello Stato Attuale del Sistema

L'analisi è stata condotta con particolare riferimento alla **qualità del software nel flusso di upload**, avvalendosi anche del supporto del tool **SonarQube**. Dallo studio dello stato attuale sono emerse le seguenti criticità:

- **Presenza diffusa di code smell e antipattern**, che compromettono la manutenibilità del codice, rendendone complessa la comprensione, l'estensione e l'evoluzione. Ciò ha reso necessario un intervento significativo di refactoring.
- **Bug applicativi** che impediscono il corretto upload delle classi e dei relativi test dei robot (dettagliati nelle sezioni successive).
- **Documentazione quasi assente**, con conseguenti difficoltà di onboarding e di comprensione del funzionamento del sistema.

Inoltre è stata definita una user story per comprendere meglio quali sono i miglioramenti richiesti dall'amministratore



2.2 Requisiti Funzionali

Dall'analisi della user story e delle criticità emerse, sono stati individuati i seguenti **requisiti funzionali**:

RF1 - bug fixing

Risolvere i problemi che compromettono la funzionalità di upload (vedi tabella punto 2.4)

RF2 - gestione degli errori verso il Front-End

Il sistema deve restituire messaggi di errore esplicativi secondo il pattern **WHAT + WHY + HOW**, al fine di migliorare la comprensione del problema e agevolare eventuali azioni correttive.

RF3 - debug mode micro servizio T7

Definire una modalità di debug che permette di preservare i file temporanei della coverage generation

RF4 - pulizia dei file in caso di errore di upload

Il sistema deve garantire la rimozione dei file temporanei o persistenti nel caso in cui l'upload fallisca, evitando situazioni in cui residui di file impediscano un nuovo tentativo di upload per la stessa classe.

RF5 - input validation

Il sistema manca di un input validation strutturato e che venga effettuato sia nel FE che nel BE

RF6 - reset del form

Dopo aver caricato una classe con successo, se l'amministratore non desidera tornare alla lista delle classi caricate è necessario resettare il form di input per evitare il ri-caricamento della stessa classe

RF7 - miglioramento della comunicazione tra i servizi

I microservizi **T7** e **T8** devono fornire risposte più chiare e informative verso **T1**, includendo messaggi di errore e informazioni di stato più dettagliate.

2.3 Requisiti Non Funzionali

Sono stati inoltre identificati i seguenti **requisiti non funzionali**, principalmente legati alla qualità del software e alla manutenibilità:

RNF1 - uniformità e chiarezza dei log

I log devono essere standardizzati e resi più leggibili, al fine di facilitare le attività di debugging, monitoraggio e manutenzione.

RNF2 - miglioramento delle qualità del software

Il codice sorgente deve essere rifattorizzato per ridurre code smell e antipattern, migliorando la leggibilità, la modularità e la separazione delle responsabilità.

RNF3 - creazione documentazione

Per quanto generalmente la creazione della documentazione non sia un vero e proprio requisito, in questo caso essendo una parte importante di questo task abbiamo comunque deciso di esplicitarlo come tale

RNF4 - controller OpenAPI Specification

introdurre OpenAPI specification per il controller di upload degli oppoenti

2.4 Bug identificati

Di seguito vengono riportati i bug che sono stati individuati nel flusso di Upload che ne compromettono le funzionalità:

ID	DESCRIZIONE	LOCALIZZAZIONE (com.groom.manvsclass...)
B01	Calcolo del coverage EvoSuite non funzionante se nessun file presente nella cartella ReportTest.	util/filesystem/ FileOperationUtil.java
B02	Rimozione invocazioni duplici EvoSuite e/o JaCoCo.	service/ UploadOpponentService.j ava
B03	Se il nome della classe indicato nel form, non equivale al nome della classe dichiarata nel sorgente java, e nell'archivio caricato non sono presenti i file di coverage, EvoSuite e JaCoCo crashano. Se invece sono presenti i file di coverage, le unità di test non	service/ OpponentService.java

	vengono compilate correttamente durante il gioco.	
B04	tag <i>expose</i> nei dockerfile deprecato	docker file di T1 e T5
B05	Error response non uniforme tra front-end e back-end.	service/ OpponentService.java

2.4 Analisi dell’Impatto

Le modifiche hanno interessato principalmente il micro servizio **T1**. In particolare:

- Le service class **OpponentService** e **UploadOpponentService**, insieme alle rispettive classi di utilità, sono stati rifattorizzati e scomposti per migliorare chiarezza, coesione e manutenibilità.
- È stata introdotta una **modalità di debugging nel micro servizio T7**, che consente la persistenza dei file temporanei utilizzati per la generazione della coverage, configurabile tramite un’apposita variabile d’ambiente.

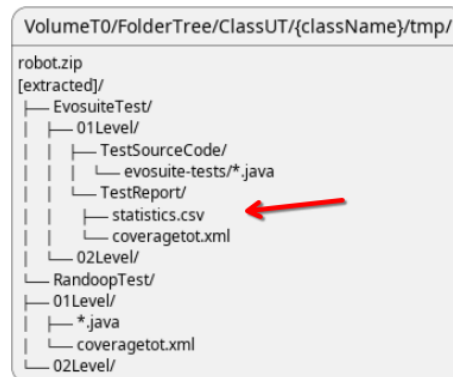
3. PROGETTAZIONE DELLA SOLUZIONE

3.1 SOLUZIONI REQUISITI FUNZIONALI

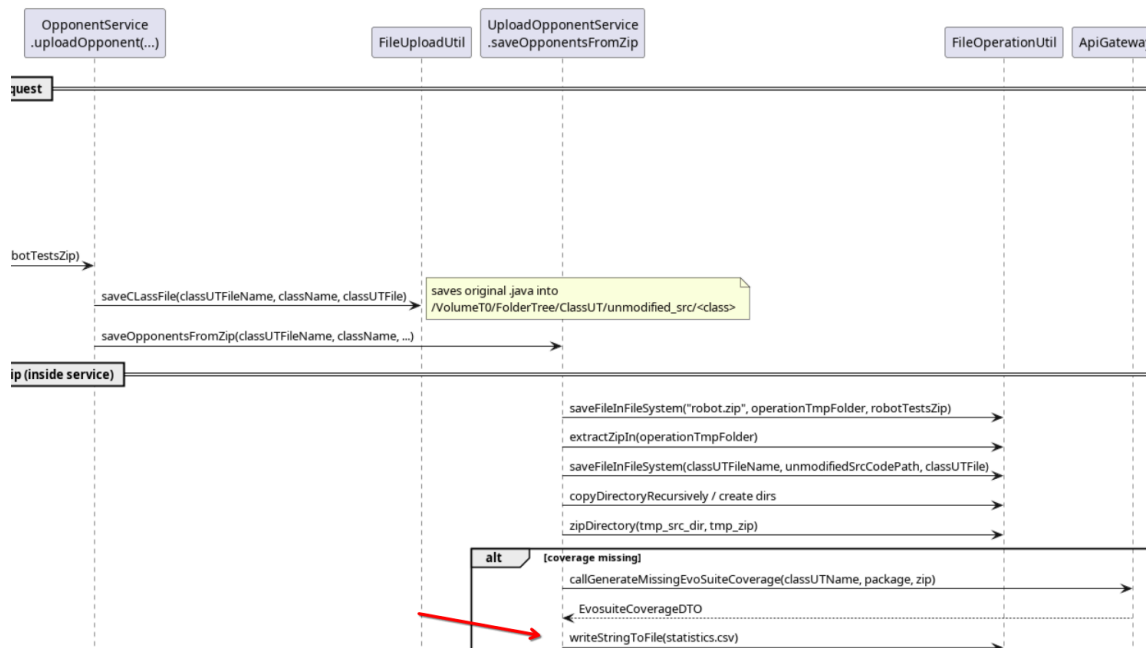
RF1 - bug fixing

- **B01 - generazione test Evosuite**

Nel caso in cui venga caricato in input un file zip **non contenente alcun file nella cartella TestReport/** nei test di Evosuite, questo comporterà un errore nella fase di upload.



Questo avviene perché in questo caso, non viene creato correttamente l'intero **albero di directory** nel volume condiviso **T0**, di conseguenza questo porta a far sì che una volta ricevuto il file di Coverage dal micro servizio **T8**, il metodo utilizzato per salvare il file non trovasse la parent directory corretta lanciando un'eccezione.

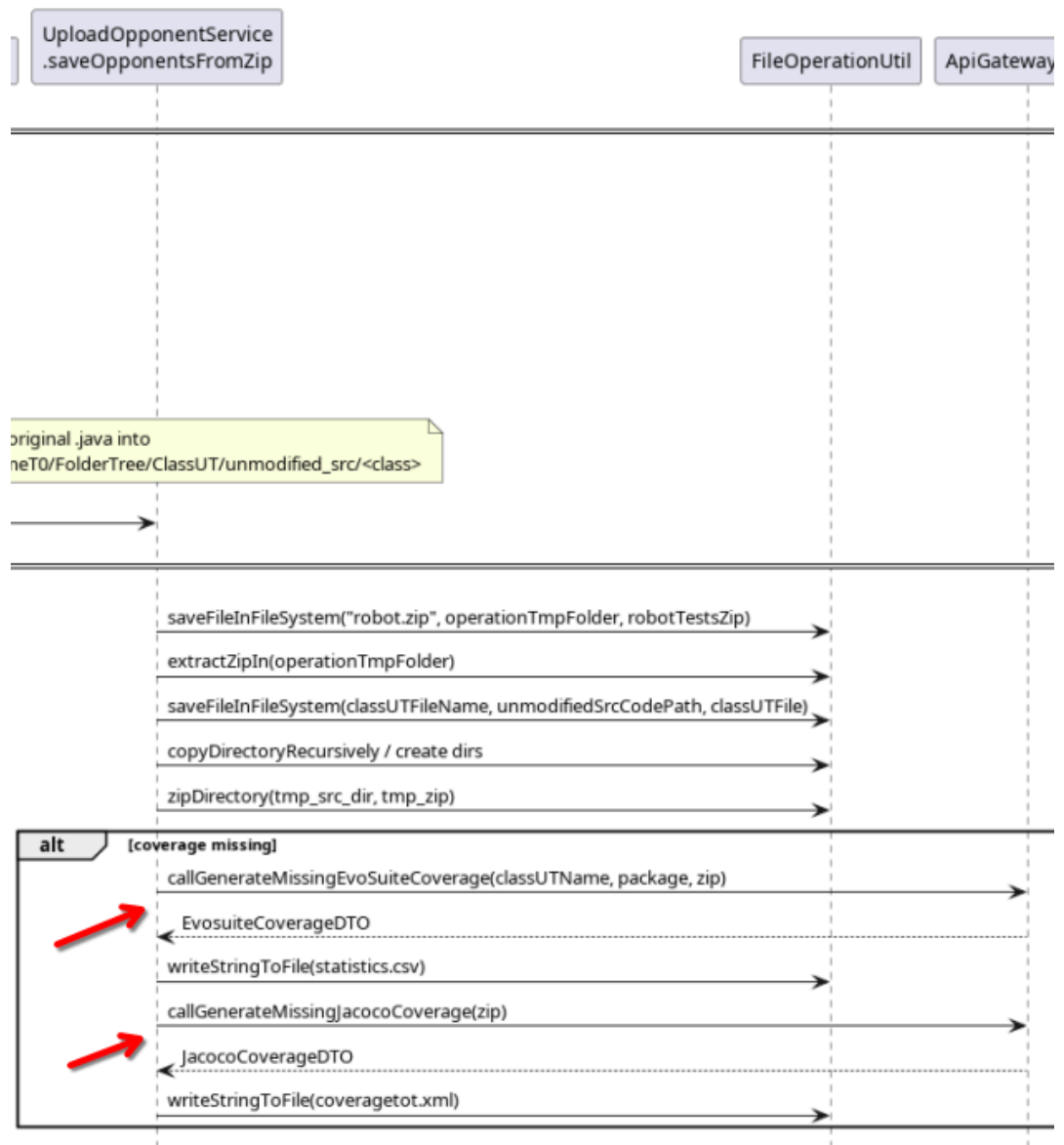


Per risolvere questo problema è stato aggiornato il metodo di salvataggio del file, che ora **controlla prima se le parent directory** sono presenti ed in caso le crea iterativamente.

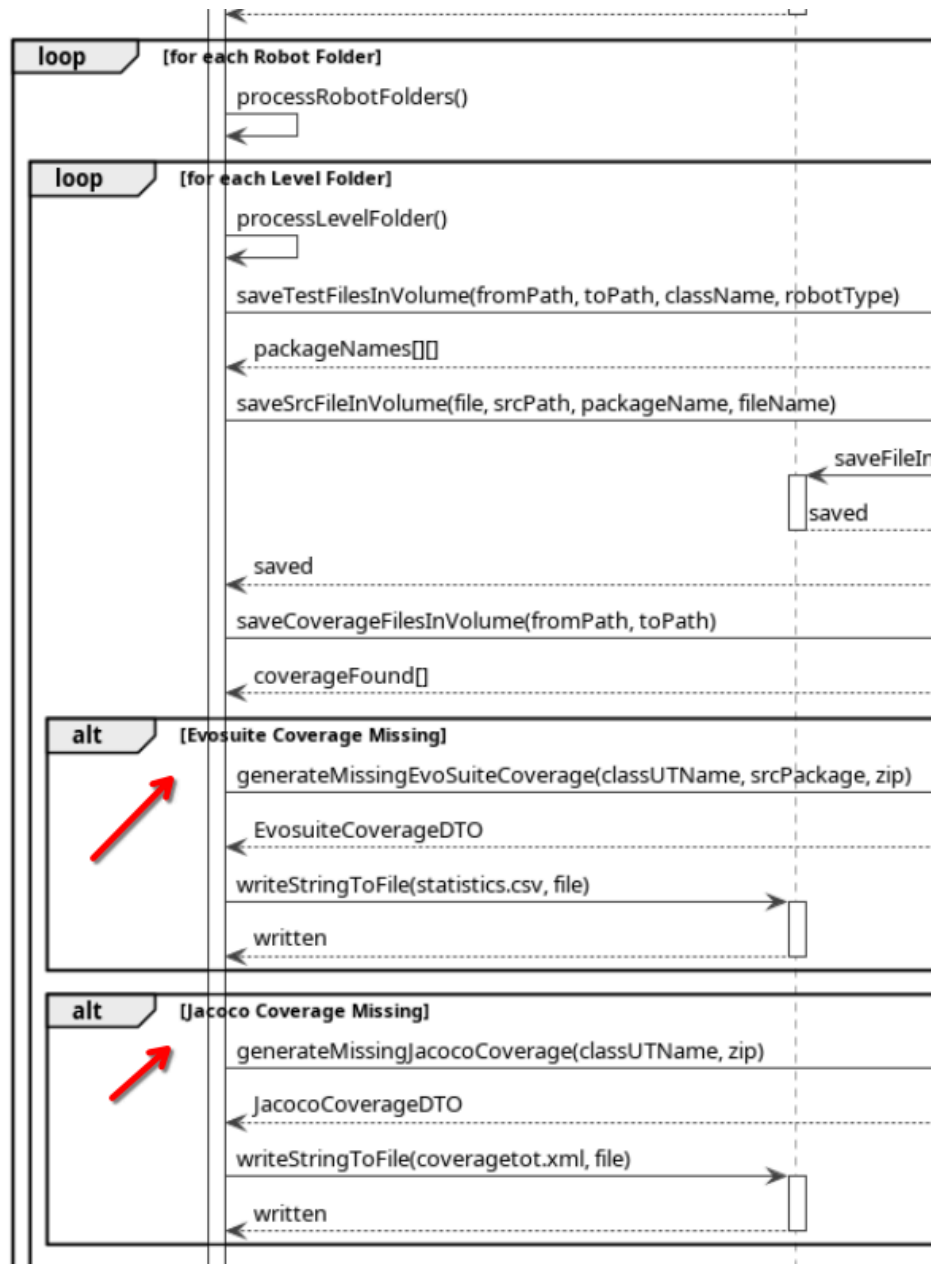
```
public void writeStringToFile(String content, File file) throws IOException {
    File parent = file.getParentFile();
    if (parent != null) {
        Files.createDirectories(parent.toPath());
    }
    try (FileWriter writer = new FileWriter(file)) {
        writer.write(content);
    }
}
```

- **B02 - duplici chiamate a Evosuite e Jacoco**

In seguito all'upload di una classe e del relativo archivio contenenti i test, se i file contenenti le metriche di uno dei due robot fossero assenti, **T1 invocherebbe sia T7** che **T8** per ricalcolare i file di coverage per entrambi i robot, invece che invocare esclusivamente la generazione del robot da cui sono stati generati i file di test.



Per risolvere questo problema, viene adesso controllato il **robotType**, in un apposito nuovo metodo, prima di invocare la coverage generation di Evosuite o Jacoco, gestendo anche **robotType** non conosciuti.



- **B03 - nome della classe nel form non corrispondente al nome classe nel file Java**

Per questa problematica sono state riscontrate due casistiche:

- **File di coverage non presenti**

Nel caso in cui non siano presenti i file di coverage, quando viene inviata la richiesta a **T7** e **T8** viene inviato anche il nome della classe, ma quando proveranno a generare i test per tale classe non riescono a trovarlo, causando un'eccezione nel flusso di Upload

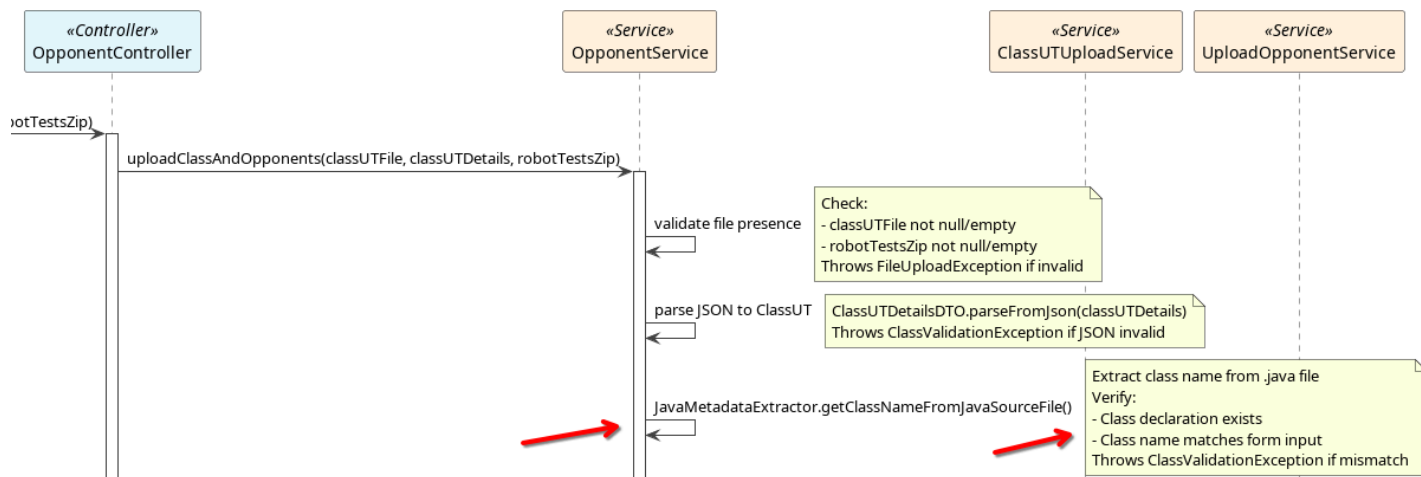
- **File di coverage presenti**

Nel caso in cui siano presenti i file di coverage, l'upload andrà a buon fine, il problema è che nella fase di gioco quando si proverà a confrontare i propri risultati con i robot, mvn andrà in errore non trovando la classe con tale nome che sarà poi anche mostrato nella console della partita

Per risolvere questo problema è stato scelto di utilizzare l'approccio che impattasse di meno anche negli altri microservizi, andando ad introdurre un **Input Validation sul nome della classe**, verificando che il nome inserito nel form corrisponda al nome nel file Java prima di procedere con l'upload.

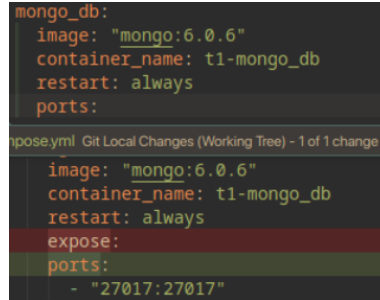
*Vedi metodo **getClassNameFromJavaSourceFile()** in*

com.groom.manvsclass.util.upload.JavaMetadataExtractor.java



- **B04 - expose tag deprecato nei dockerfile**

In seguito a un recente aggiornamento di Docker, i **DockerFile** di T1 e T5 presentano problemi di retrocompatibilità con la keyword **expose**, impedendo il deploy dei micro servizi, che è stata adesso aggiornata con **ports**.



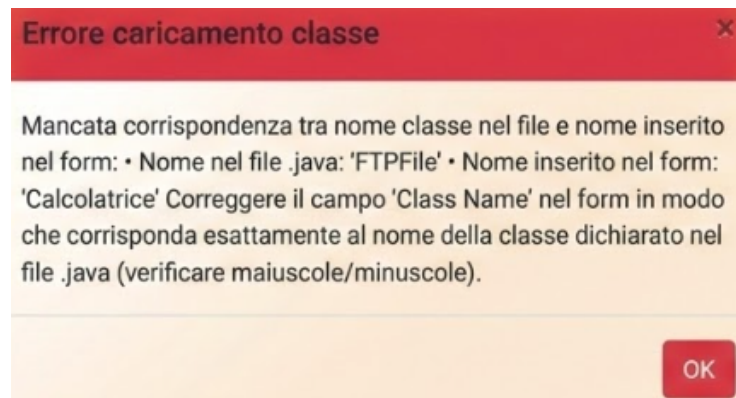
```
mongo_db:
  image: "mongo:6.0.6"
  container_name: t1-mongo_db
  restart: always
  ports:

expose.yml  Git Local Changes (Working Tree) - 1 of 1 change
image: "mongo:6.0.6"
container_name: t1-mongo_db
restart: always
expose:
ports:
  - "27017:27017"
```

RF2 - miglioramento messaggi al FE (pattern WHAT + WHY + HOW)

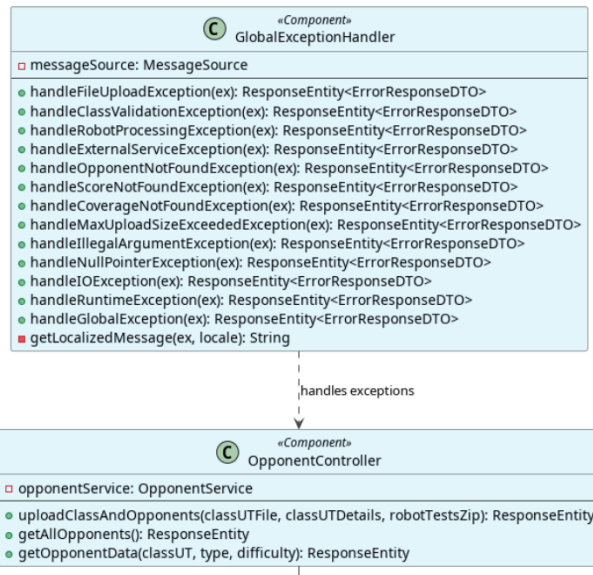
- **Nuovo modal per mostrare errori dettagliati al front-end**

In caso di errore in fase di upload, all'amministratore viene mostrato nel browser un modale personalizzato che indica le cause dell'errore.



- **Global Exception Handler**

Seguendo quanto fatto nel micro servizio **T4**, è stato introdotto una nuova classe **GlobalExceptionHandler** che intercetta tutti gli errori nel micro servizio, per uniformare la risposta ritornata attraverso la nuova **ErrorResponseDTO** e **localizzare** i messaggi in base alla lingua dell'utente, prendendo tali messaggi dagli appositi file **.properties**.
messaggi presenti in
T1-G11/applicazione/manvsclass/src/main/resources/lang/.properties*



```

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class ErrorResponseDTO {
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;
    private String path;

    public ErrorResponseDTO(int status, String error, String message) {
        this.timestamp = LocalDateTime.now();
        this.status = status;
        this.error = error;
        this.message = message;
    }

    public ErrorResponseDTO(int status, String message) {
        this.timestamp = LocalDateTime.now();
        this.status = status;
        this.error = getErrorName(status);
        this.message = message;
    }

    private String getErrorName(int status) {
        return switch (status) {
            case 400 -> "Bad Request";
            case 404 -> "Not Found";
            case 500 -> "Internal Server Error";
            default -> "Error";
        };
    }
}
  
```

- **Aggiunti controlli per errori durante l'upload**
per poter utilizzare al meglio il GlobalExceptionHandler e fornire dei messaggi più dettagliati, sono stati introdotti nuovi controlli che lanciano delle exception personalizzate, elencati di seguito:

Categoria	Fase	Descrizione dell'Errore	Eccezione Generata
Elaborazione File	Estrazione ZIP	Impossibile estrarre l'archivio ZIP	FileUploadException
Elaborazione File	Estrazione ZIP	Archivio ZIP vuoto o senza cartelle dei robot	RobotProcessingException
Elaborazione File	Estrazione ZIP	Struttura ZIP corrotta o non conforme	FileUploadException
Elaborazione File	Cartelle robot	Nome cartella robot non valido (non termina con "Test")	RobotProcessingException
Elaborazione File	Cartelle robot	Formato cartella di livello non valido (<code>\d{2,}Level</code>)	RobotProcessingException
Elaborazione File	Cartelle robot	Assenza di file di test nella cartella di livello	RobotProcessingException
Elaborazione File	File I/O	Impossibile leggere il file della classe UT	FileUploadException
Elaborazione File	File I/O	Errore nel salvataggio dei file su filesystem	FileUploadException
Elaborazione File	File I/O	Impossibile creare le directory necessarie	IOException
Generazione Coverage	EvoSuite	Fallimento chiamata API EvoSuite	RobotProcessingException
Generazione Coverage	EvoSuite	Formato non valido del file <code>statistics.csv</code>	IOException
Generazione Coverage	EvoSuite	Impossibile scrivere il file di coverage	IOException
Generazione Coverage	JaCoCo	Fallimento chiamata API JaCoCo	RobotProcessingException
Generazione Coverage	JaCoCo	Formato non valido del file <code>coveragetot.xml</code>	IOException
Generazione Coverage	JaCoCo	Impossibile scrivere il file di coverage	IOException
Persistenza	Database	Errore nel salvataggio dei metadati della Class UT	Eccezione di database
Persistenza	Database	Errore nel salvataggio dell'Opponent	Eccezione di database
Persistenza	Database	Errore durante la	Loggato (non rilanciato)

		cancellazione in rollback	
Servizi Esterni	Integrazione	Fallimento notifica all'API Gateway	ExternalServiceException
Servizi Esterni	Integrazione	Fallimento notifica all'API Gateway	ExternalServiceException

- **Miglioramento log**

I messaggi di log interni sono stati resi più dettagliati.

RF3 - modalità debug micro servizio T7

Introdotta una modalità di debug in **T7** attivabile tramite apposita variabile d'ambiente, che permette di **preservare i file temporanei** che vengono elaborati per generare il file di coverage.

controllo variabile d'ambiente applicato nel metodo `compileAndTestEvoSuiteTests()` della classe `CompilationService` di T7

```
name: t7

<> Run All Services
services:
  <> Run Service
  app:
    image: mick0974/a13:t7-g31
    container_name: t7-controller
    restart: always
    environment:
      OTEL_EXPORTER_OTLP_ENDPOINT: "http://otel-
      OTEL_EXPORTER_OTLP_PROTOCOL: grpc
      DEBUG_PRESERVE_TMP: true
```

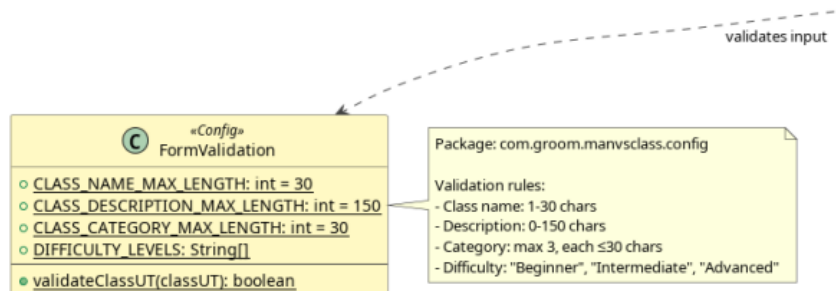
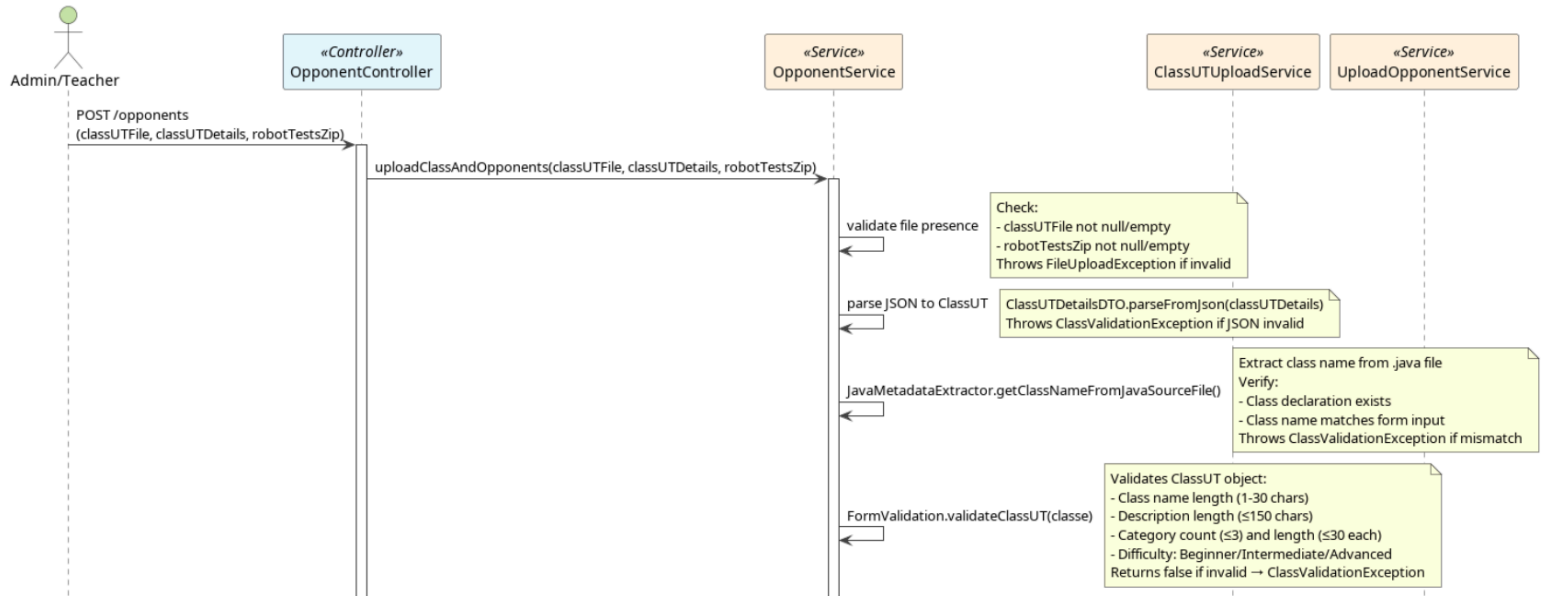
```
boolean preserveTmp = "true".equalsIgnoreCase(System.getenv("DEBUG_PRESERVE_TMP"));
if (preserveTmp) {
  logger.warn("[Compilation Service] DEBUG_PRESERVE_TMP=true, preserving compiler working dir: {}", config.getPathCompiler());
} else {
  FileUtils.deleteDirectoryRecursively(Paths.get(config.getPathCompiler()));
}
```

RF4 - clean-up file in caso di errore

Al fine di implementare una soluzione per il problema dei **file residui sul volume docker** da ripulire, è stata necessaria una **review del codice sorgente** del modulo **T1**, al fine di **ripercorrere tutte le operazioni su filesystem** che vengono svolte a ogni tentativo di upload di una classe (creazione di cartelle, estrazione di archivi zip, copia di file).

Questo perché è stato introdotto un metodo di **rollback** che tenta di eliminare tutti i possibili file e dati creati in caso di errore, senza dover tener traccia di cosa sia stato già effettivamente creato.

Vedi sequence diagram in T1-G11/documentation/diagram/final e metodo `performRollback()` in `com.groom.manvsclass.service.OpponentService.java`



In particolare l'elenco dei controlli è:

INPUT	SIDE	CONDITIONS
className	FE & BE	non-empty; len 1–30; equals .java (case-insensitive); extracted .java class name == form name (case-insensitive)
classUTFile	FE & BE	present; not empty; readable; contains class declaration
classUTFileName	BE	cleaned path
robotTestsZip	FE & BE	present; not empty; extracts; contains robot-group folder

INPUT	SIDE	CONDITIONS
classUTDetails	BE	JSON parseable
difficulty	BE	one of {Beginner,Intermediate,Advanced}
description	FE & BE	if present len <= 150
category	FE & BE	if present size <= 3; each len <= 30
robot-group folders	BE	name ends with "Test"
robotType	BE	folder name without "Test"
levelFolders	BE	match \d{2,}Level
testFiles	BE	at least one .java per level

RF6 - reset del form HTML dopo un upload

In seguito a un upload corretto di una classe, l'intero **form HTML** viene **resettato**, in modo da impedire un accidentale re-upload della stessa classe da parte dell'amministratore.

*reset effettuato nel metodo **uploadOpponent()** nel file javascript `resources/static/t1/js/opponents/opponents_upload.js`*

```
$('#successModal').modal('show');
//-----
// Se la classe è stata caricata correttamente, allora resetta il form.
const form = classInput?.form;
if (form) {
  form.reset();
}
```

3.2 SOLUZIONI REQUISITI NON FUNZIONALI

RNF1 - uniformità e chiarezza dei log

Il sistema di **log** interni in alcuni punti risulta non uniformato con chiamate che redirezionano output testuale verso lo standard output (usando **System.out.println**), adesso standardizzati con **SLF4J**.

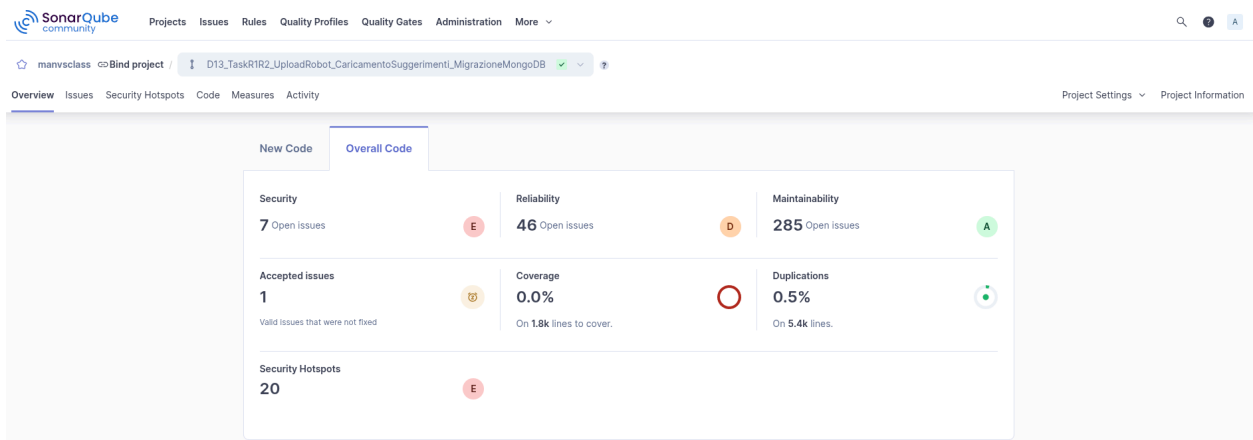
```
public ResponseEntity<FileUploadResponse>
    uploadOpponent(...) {
    ...
    // None del file e dimensione
    String classUTFileName = StringUtils.cleanPath(...);
    long size = classUTFile.getSize();
    System.out.println("Salvataggio di " +
classUTFileName + " nel filesystem condiviso");
}
```

```
public void saveClassUTFile(String classUTFileName,
    String classUTName, MultipartFile classUTFile)
{
    logger.info("Salvataggio di {} nel filesystem
condiviso", classUTFileName);
}
```

RNF4 - miglioramento delle qualità del software

Il codice sorgente presentava numerosi punti contrassegnati da **SonarQube** come code smells nel flusso di upload, che sono stati completamente rimossi nella seconda iterazione.

(approfondito nella sezione 4: struttura del progetto aggiornata)



RNF5 - creazione documentazione

Sono stati generati i **class diagram** e i **sequence diagram** del progetto, sia prima dell'inizio del task, e in seguito alla terza iterazione.

Inoltre sono stati realizzati degli activity diagrammi che spiegano il flusso di elaborazione dell'input e lo stato del file system step dopo step.

Presenti nella cartella T1-11/documentation/diagrams/final e mostrati alla fine di questo documento

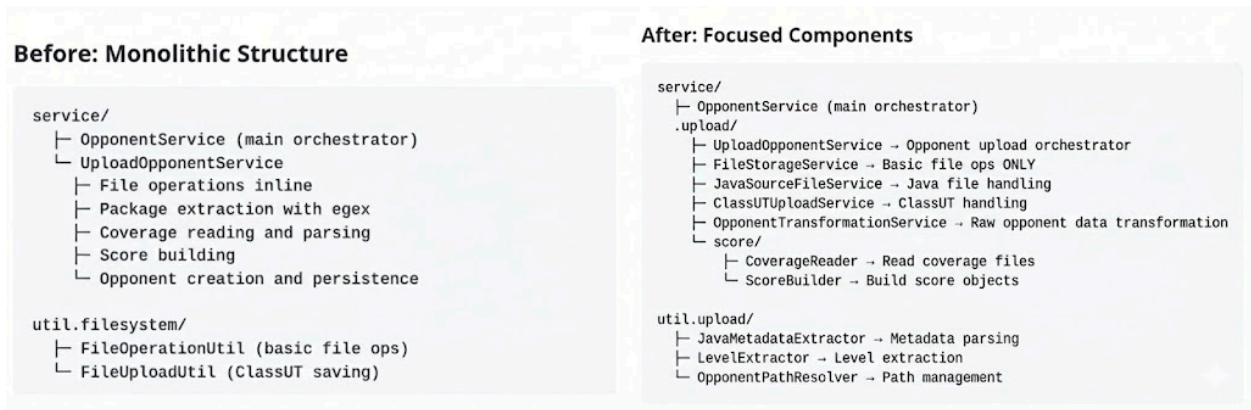
RNF6 - controller OpenAPI Specification

Aggiunta l'OpenAPI specification al controller di upload opponent per indicare i codici di risposta e le informazioni aggiuntive a riguardo.

```
public class OpponentController {
    @PostMapping("")
    @Operation(
        summary = "Upload class under test and robot tests",
        description = "Uploads a Java class file (ClassUT) along with its metadata and robot-generated test suites (EvoSuite/Randoop). " +
            "The endpoint validates the class file structure, verifies class name consistency, processes robot test archives, " +
            "and stores opponents with different difficulty levels. The robot tests zip must contain folders named 'EvoSuiteTest' " +
            "and/or 'RandoopTest' with subdirectories for each difficulty level (e.g., 01Level, 02Level) containing test files and coverage reports."
    )
    @ApiResponse(value = {
        @ApiResponse(
            responseCode = "200",
            description = "Successfully uploaded and processed class and robot tests. Returns metadata about the uploaded files including file names, URIs, and sizes.",
            content = @Content(
                mediaType = "application/json",
                schema = @Schema(implementation = FileUploadResponse.class)
            )
        ),
        @ApiResponse(
            responseCode = "400",
            description = "Bad Request - Validation errors occurred. Possible causes: " +
                "empty or missing files, class name mismatch between form and source file, " +
                "invalid Java syntax, missing package declaration, invalid ZIP structure, " +
                "missing robot test directories (EvoSuiteTest/RandoopTest), or invalid test file format.",
            content = @Content(
                mediaType = "application/json",
                schema = @Schema(implementation = ErrorResponseDTO.class)
            )
        ),
        @ApiResponse(
            responseCode = "409",
            description = "Conflict - A class with the same name already exists in the system.",
            content = @Content(
                mediaType = "application/json",
                schema = @Schema(implementation = ErrorResponseDTO.class)
            )
        ),
        @ApiResponse(
            responseCode = "500",
            description = "Internal Server Error - Unexpected error occurred during file processing. " +
                "ZIP extraction, database operation, or robot test analysis.",
            content = @Content(
                mediaType = "application/json",
                schema = @Schema(implementation = ErrorResponseDTO.class)
            )
        )
    })
    public ResponseEntity<FileUploadResponse> uploadClassAndOpponents(
```

4. IMPLEMENTAZIONE E STRUTTURA DEL PROGETTO AGGIORNATO

Di seguito è riportata una riorganizzazione e riformulazione delle classi e delle loro responsabilità all'interno del micro servizio ed i rispettivi package in forma tabellare, nei punti successivi sono disponibili anche gli appositi diagrammi



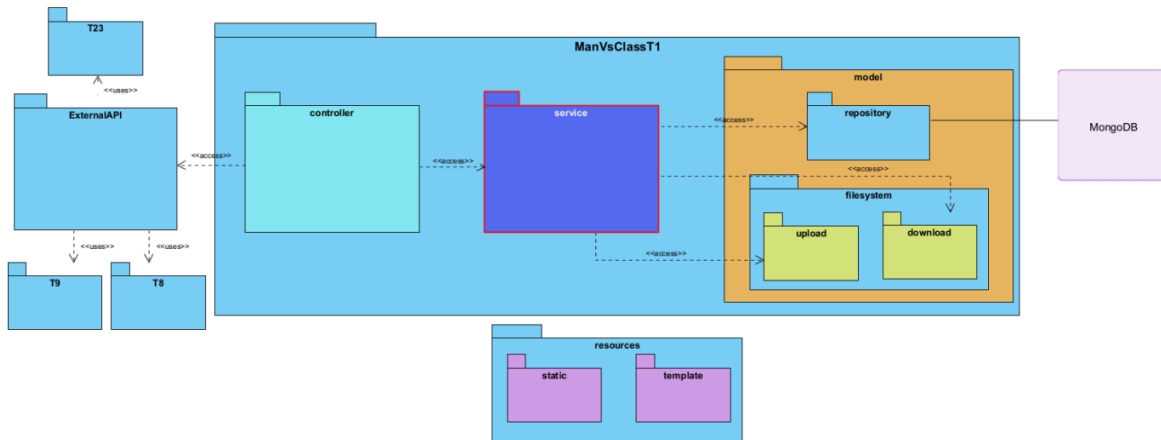
Package	Classe	Descrizione
com.groom.manvsclass		package parent di tutti i package
service	OpponentService	Coordina il flusso principale per il caricamento (upload) sia della Classe Sotto Test che degli <i>opponent</i> .
service.upload	UploadOpponentService	Gestisce l'orchestrazione specifica del caricamento dei robot (<i>opponent</i>).
service.upload	ClassUTUploadService	Si occupa della logica di gestione del caricamento della classe sotto test.
service.upload	FileStorageService	Esegue le operazioni di gestione (lettura, scrittura, ecc.) dei file persistenti nel file system.
service.upload	JavaSourceFileService	Fornisce servizi per la gestione dei file sorgente Java.

service.upload ▾	OpponentTransformationService	Effettua la trasformazione e la conversione dei dati dei robot nelle strutture dati utilizzate per la persistenza.
service.upload.score ▾	CoverageReader	Responsabile della lettura e dell'estrazione dei dati dai file di <i>coverage</i> .
service.upload.score ▾	ScoreBuilder	Genera i dati relativi allo <i>score</i> a partire dalle metriche estratte dai file di <i>coverage</i> .
util.upload ▾	JavaMetadataExtractor	Estrae i metadati essenziali dai file Java (ad esempio, il <i>package name</i>).
util.upload ▾	LevelExtractor	Determina e ricava i livelli associati ai robot per i quali sono stati caricati i test.
util.upload ▾	OpponentPathResolver	Gestisce la risoluzione dei percorsi (<i>path management</i>), inclusa l'applicazione di espressioni regolari (regex) per l'estrazione da file compressi (<i>zip</i>).
controller ▾	GlobalExceptionHandler	Fornisce la gestione centralizzata di tutte le eccezioni che si verificano all'interno del micro servizio.
config ▾	FormValidation	Implementa la logica di validazione per l'input dati nel flusso di caricamento (<i>upload</i>).

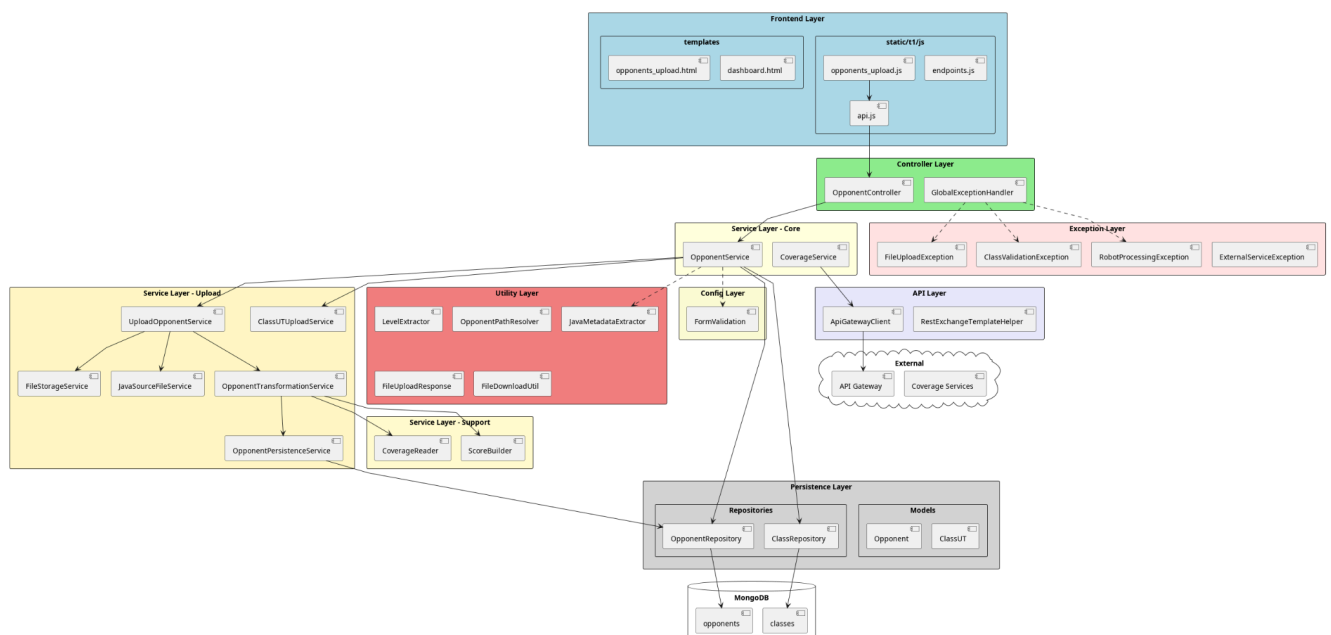
4.2 DIAGRAMMI AGGIORNATI

4.2.1 PACKAGE DIAGRAM (IBRIDO CON CLASS DIAGRAM)

PROGETTO INIZIALE:

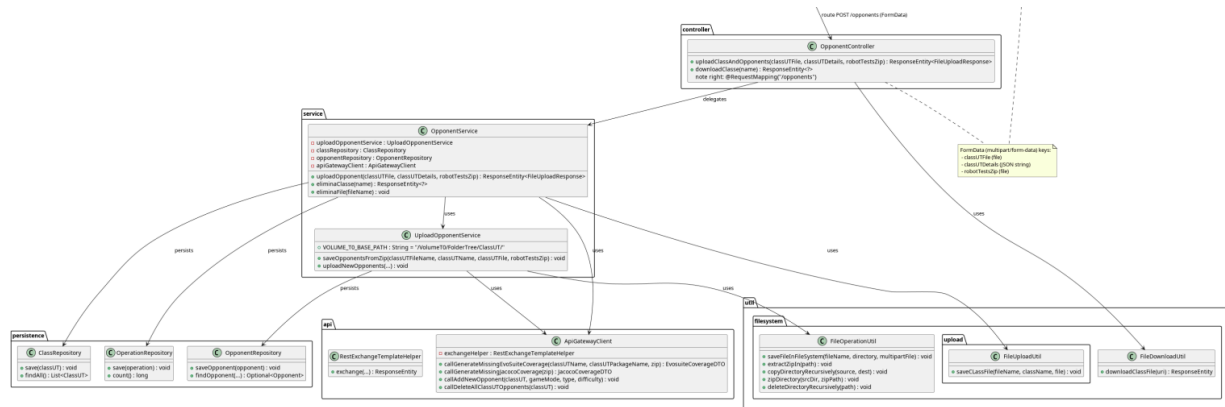


PROGETTO CONSEGNATO:

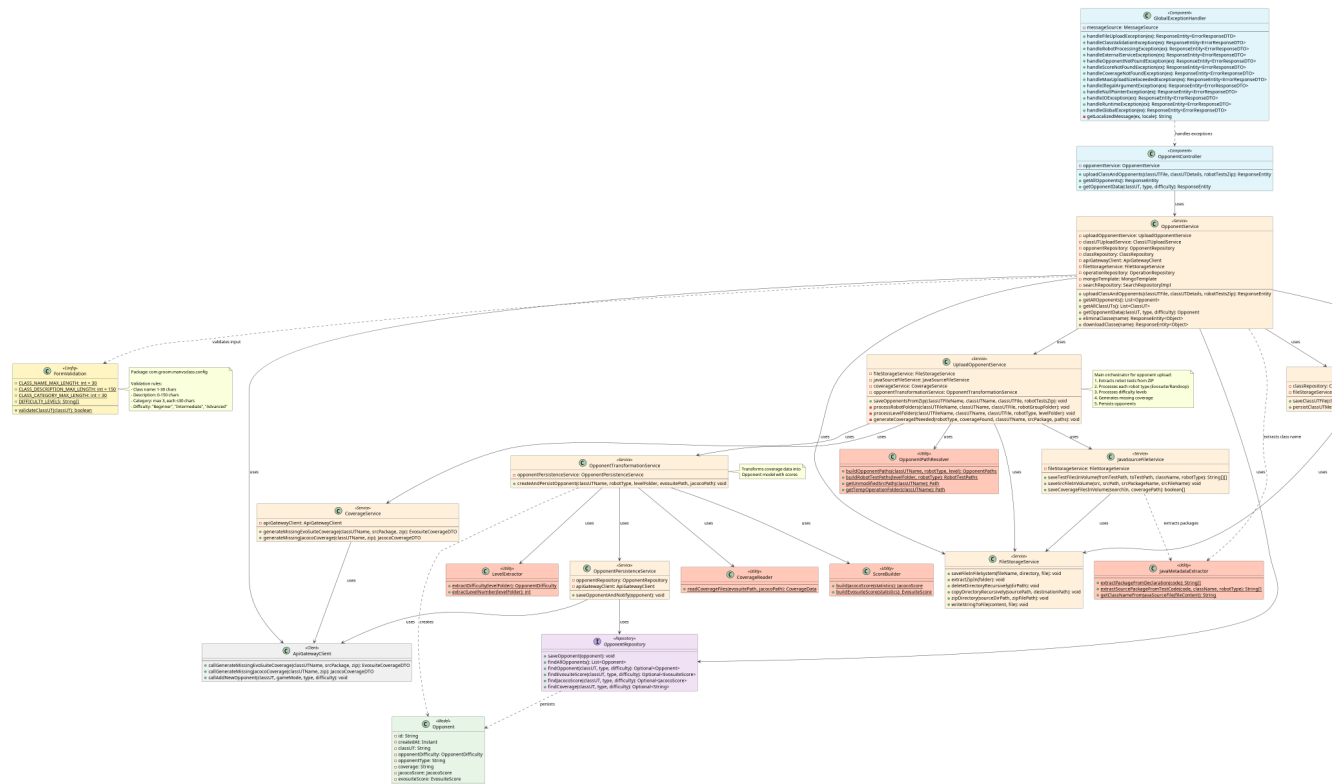


4.2.2 CLASS DIAGRAM

PROGETTO INIZIALE:

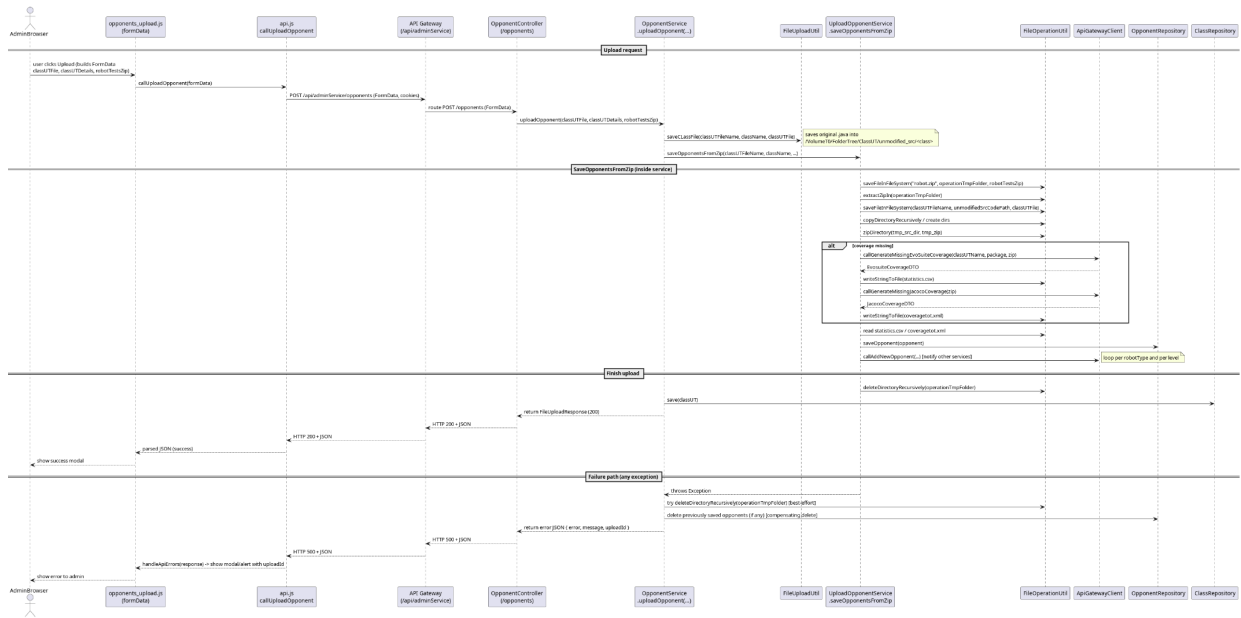


PROGETTO CONSEGNATO:

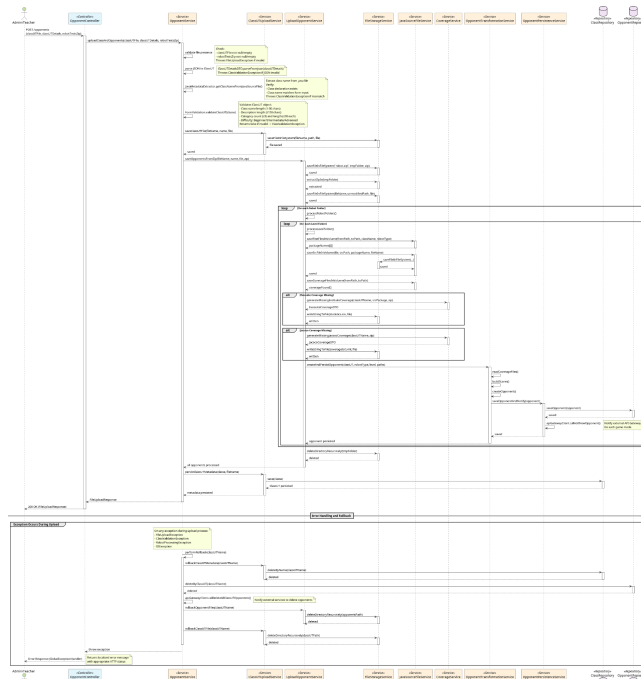


4.2.3 SEQUENCE DIAGRAM

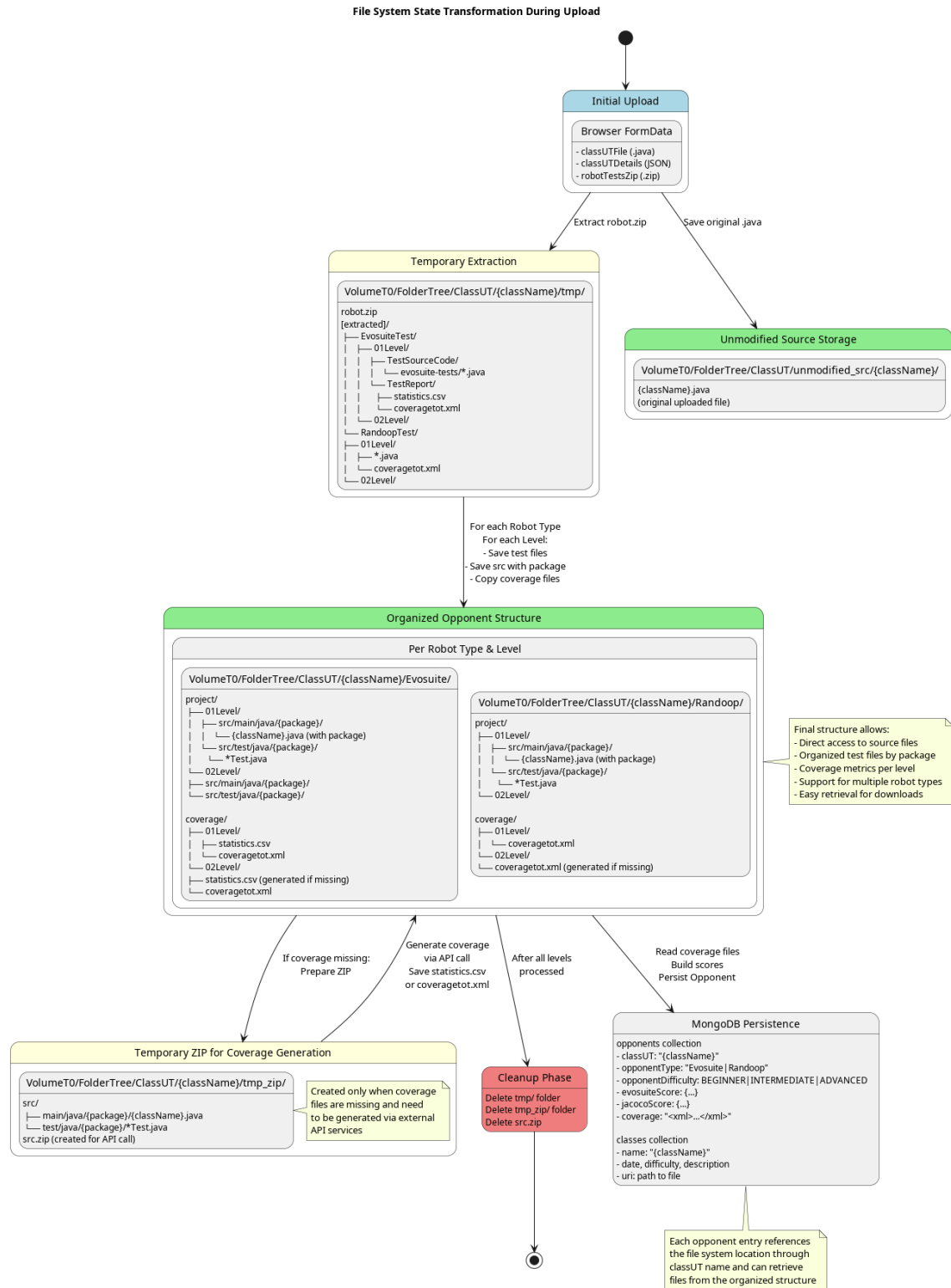
PROGETTO INIZIALE:



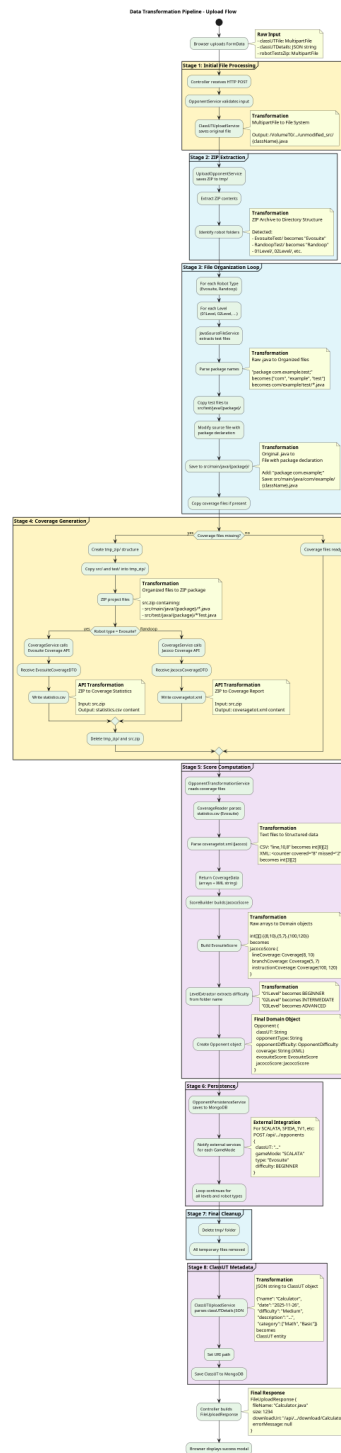
PROGETTO CONSEGNATO:



4.2.4 FILE SYSTEM STATE ACTIVITY DIAGRAM



4.2.5 DATA TRANSFORMATION PIPELINE ACTIVITY DIAGRAM



5. DESCRIZIONE DEI TEST EFFETTUATI

I test sono stati divisi in due categorie, ovvero quelli inerenti al contenuto del file .zip dei test della classeUT e quelli inerenti agli altri campi del form.

5.1 test sul file .zip contenente i test

Tutti i file zip di questa sezione di test sono presenti nella cartella
<HOME_REPO>/ClassiUT/Classi2025/OutputFormat_wrong_input_test/
e considerano come input nel form:

- ☐ Class name: OutputFormat
- ☐ Livello: facile
- ☐ Java file: OutputFormat.java presente nella stessa cartella dei test

ID Test	Nome test e file zip	Cosa Testa	Comportamento Atteso	Codice Errore	Punto di Validazione
T001	only_evosuite_no_coverage	Test EvoSuite senza file di copertura	Il sistema genera la copertura EvoSuite mancante	Generazione copertura con successo	UploadOpponentService.ensureEvoCoverageIfMissing()
T002	no_coverage	Test EvoSuite e Randoop senza file di copertura	Il sistema genera i file di copertura mancanti	Generazione copertura con successo	UploadOpponentService.coverage methods
T003	invalid_folder_name	Cartella robot "InvalidFolderName" non termina con "Test"	Cartella ignorata → nessun robot valido	error.robot.folder.noValid	UploadOpponentService.isValidRobotFolder()
T004	invalid_level_name	Cartella livello "InvalidLevel" non corrisponde a \d{2,}Level	Livello ignorato → nessun livello valido	error.robot.level.noValid	UploadOpponentService.isValidLevelFolder()
T005	no_java_files	Cartella livello contiene solo test.txt (nessun .java)	Nessun file di test rilevato	error.robot.noTests	UploadOpponentService.isValidTestFolder()

T006	level_single_digit	Cartella livello denominata "1Level" invece di "01Level"	Livello ignorato → nessun livello valido	error.robot.level.noValid	UploadOpponentService.isValidLevelFolder()
T007	tests_not_in_level	File di test direttamente nella cartella EvoSuiteTest	Nessuna cartella livello valida trovata	error.robot.level.noValid	UploadOpponentService.uploadNewOpponents()
T008	wrong_root_folder	ZIP contiene "notrobots/" invece di "robots/"	Prima cartella presa ma nessun robot valido	error.robot.folder.noValid	UploadOpponentService.processRobotFolders()
T009	empty_robots_folder	Cartella robots/ esiste ma è completamente vuota	Nessuna cartella tipo-robot trovata	error.robot.folder.empty	UploadOpponentService.processRobotFolders()
T010	empty_level_folders	Struttura EvoSuiteTest/01Level valida ma vuota	Nessun file di test in alcun livello	error.robot.noTests	UploadOpponentService.processLevelFolder()
T011	folder_not_ending_test	Cartella RandomTest (pattern di denominazione valido)	Dovrebbe elaborare con successo	Successo (200 OK)	Test positivo - tutte le validazioni passano
T012	robot_no_test_suffix	Cartella "Robot" senza suffisso "Test"	Cartella ignorata → nessun robot valido	error.robot.folder.noValid	UploadOpponentService.isValidRobotFolder()
T013	nested_test_files	File di test in 01Level/SubFolder invece di 01Level	Livello appare vuoto (nessun .java diretto)	error.robot.noTests	UploadOpponentService.isValidTestFolder()

T014	both_tests_only_jacoco	EvoSuite+Randooop con solo copertura Jacoco	Il sistema genera la copertura EvoSuite mancante	Generazione copertura con successo	UploadOpponentService.ensureEvoCoverageIfMissing()
T015	full_tests_no_jacoco	EvoSuite con copertura Evosuite ma senza Jacoco	Il sistema genera la copertura Jacoco mancante	Generazione copertura con successo	UploadOpponentService.ensureJacocoCoverageIfMissing()
T016	bad_input_missing_file	EvoSuite+Randooop ma Randooop senza file di test	Elabora EvoSuite con successo ma non Jacoco	error.robot.noTests	UploadOpponentService.processLevelFolder()
T017	empty_root_only	ZIP con solo cartella robots/ (nessuna sottocartella)	Nessuna cartella tipo-robot trovata	error.robot.folder.empty	UploadOpponentService.processRobotFolders()
T018	empty_folders_only	Struttura completa ma tutte le cartelle vuote	Nessun file di test trovato da nessuna parte	error.robot.noTests	UploadOpponentService.processLevelFolder()
T019	corrupted_zip	File ZIP non valido che non può essere estratto	Estrazione ZIP fallisce con IOException	error.robot.zip.extract	FileStorageService.extractZipIn()

5.1 Test sugli altri campi del form di input

ID Test	Tipo di Test	Cosa Testare	Comportamento atteso	Codice Errore	Punto di validazione
M01	classUTFFile vuoto	Caricamento senza selezionare file classe	Frontend blocca il caricamento	N/A (lato client)	opponents_upload.js
M02	robotTestsZip vuoto	Caricamento senza selezionare file ZIP	Frontend blocca il caricamento	N/A (lato client)	opponents_upload.js
M03	className vuoto	Caricamento con campo nome classe vuoto	Frontend blocca il caricamento	N/A (lato client)	opponents_upload.js
M04	JSON non valido	Invio dati form corrotti	Parsing JSON fallisce	error.upload.json.parse	OpponentService.uploadClassAndOpponents()
M05	Nessuna dichiarazione classe	File .java senza parola chiave <code>class</code>	Validazione fallisce	error.validation.class.noDeclaration	OpponentService.uploadClassAndOpponents()
M06	Nome classe non corrispondente	Nome form $\neq$ nome classe .java	Validazione fallisce	error.validation.class.nameMismatch	OpponentService.uploadClassAndOpponents()
M07	Dati form non validi	Nome troppo lungo / difficoltà non valida	Validazione form fallisce	error.validation.class.failed	FormValidation.validateClassUT()
M08	Descrizione troppo lunga	Descrizione > 150 caratteri	Validazione form fallisce	error.validation.class.failed	FormValidation.validateClassUT()

M09	Categoria troppo lunga	Singola categoria > 30 caratteri	Validazione form fallisce	error.validation.class.failed	FormValidation.validateClassUT()
M10	Troppe categorie	Più di 3 categorie	Validazione form fallisce	error.validation.class.failed	FormValidation.validateClassUT()

6. SVILUPPI FUTURI / RACCOMANDAZIONI

- Requisito RF7 (miglioramento log e risposte da **T7** e **T8**, poco parlanti e in alcune situazioni ambigue (es. http code 20x anche in caso di errore)) non soddisfatto in quanto richiede un’analisi più approfondita di tali micro servizi con conseguente refactoring
- Creazione Wiki